

UNIVERSIDAD TÉCNICA ESTATAL DE QUEVEDO

Facultad de Ciencias de la Computación y Diseño Digital
Carrera de Ingeniería en Software

LOGOTIPO
UTEQ

GUÍA Y EVIDENCIA DE EXPOSICIÓN–DEMOSTRACIÓN

Generación de código fuente a partir de modelos UML mediante StarUML

Proyecto Fin de Curso

Sistema de Gestión para un Gimnasio

Subsistema: Gestión de clientes, membresías y pagos

Asignatura: Ingeniería de Requisitos (ISR-401)
Docente: Ing. Gleiston Guerrero Ulloa, PhD.
Periodo académico: 2026–2027 PPA
Paralelo: 4.º Software “A”

Integrantes y roles

Integrante	Rol
Díaz Pontón Steven Santiago	[Rol por definir]
Escudero Plaza María del Rosario	[Rol por definir]
Mera Arias Erick Jhair	[Rol por definir]
Mesías Quijije Jhon Alexander	[Rol por definir]

Repositorio público de GitHub

https://github.com/Erick-Mera/GA_Exposicion_Segundo_Corte

Quevedo – Ecuador
2026

Índice

Resumen	1
1 Marco teórico	1
1.1 Model-Driven Engineering (MDE)	1
1.2 Model-Driven Architecture (MDA)	1
1.3 Model-Driven Development (MDD)	1
1.4 Niveles de abstracción	1
1.4.1 Computation Independent Model (CIM)	1
1.4.2 Platform Independent Model (PIM)	2
1.4.3 Platform Specific Model (PSM)	2
1.5 Transformaciones de modelos	2
1.5.1 Transformaciones modelo a modelo (M2M)	2
1.5.2 Transformaciones modelo a texto (M2T)	2
1.6 Metamodelos	2
1.7 Perfiles UML, estereotipos y valores etiquetados	2
1.8 Plantillas de generación	2
1.9 Forward, reverse y round-trip engineering	3
1.10 Diagramas UML como origen de generación	3
1.11 Estado del arte de las herramientas de transformación	3
2 Justificación de la selección de StarUML	3
2.1 Requisitos y restricciones tecnológicas del PFC	3
2.2 Criterios de selección de la herramienta	3
2.3 Compatibilidad de StarUML con el lenguaje y la plataforma objetivo	3
2.4 Licencia y disponibilidad	4
2.5 Curva de aprendizaje	4
2.6 Comparación con una herramienta alternativa	4
2.7 Decisión de selección	4
3 Descripción del subsistema del PFC	4
3.1 Contexto del Sistema de Gestión para un Gimnasio	4
3.2 Descripción del subsistema de gestión de clientes, membresías y pagos	5
3.3 Actores relacionados	5
3.4 Requisitos funcionales relevantes	5
3.5 Procesos principales	5

3.6	Alcance del subsistema	5
3.7	Exclusiones del subsistema	5
4	Modelo UML de origen	6
4.1	Descripción general del modelo	6
4.2	Diagrama de clases	6
4.3	Clases del dominio	6
4.4	Atributos y operaciones	6
4.5	Relaciones y cardinalidades	6
4.6	Esteretipos y perfiles utilizados	7
4.7	Validación sintáctica y semántica del modelo	7
4.8	Archivo nativo del modelo en el repositorio	7
5	Configuración del generador	7
5.1	Generador o extensión utilizada	7
5.2	Origen y versión del generador	7
5.3	Lenguaje y plataforma objetivo	7
5.4	Parámetros de generación	7
5.5	Directorio de salida y convenciones de nombres	8
5.6	Política ante la regeneración	8
5.7	Decisiones de mapeo UML a código	8
5.8	Archivos de configuración en el repositorio	8
6	Proceso de generación	8
6.1	Preparación del entorno	8
6.2	Apertura y selección del modelo de origen	9
6.3	Ejecución del generador	9
6.4	Registro o log de salida	9
6.5	Árbol del proyecto generado	9
6.6	Artefactos producidos	9
6.7	Tiempo de generación	9
7	Verificación del código generado	10
7.1	Estructura del proyecto generado	10
7.2	Verificación sintáctica	10
7.3	Compilación del código	10
7.4	Ejecución de una unidad mínima demostrable	11

7.5	Correspondencia entre modelo y código	11
7.6	Diferenciación entre código generado y código manual	11
7.7	Limitaciones del generador	11
8	Cierre del ciclo y roundtrip controlado	12
8.1	Estado inicial del modelo y del código	12
8.2	Cambio realizado en el modelo UML	12
8.3	Regeneración del código	12
8.4	Evidencia antes y después	12
8.5	Trazabilidad modelo a código	12
8.6	Manejo del código manual ante la regeneración	12
9	Reparto del trabajo	12
9.1	Roles y responsabilidades	12
9.2	Tareas realizadas por integrante	13
9.3	Commits atribuibles a cada integrante	13
9.4	Log de aportaciones de GitHub	13
9.5	Participación en la exposición y demostración	13
10	Conclusiones	13
10.1	Valor de MDD para el Proyecto Fin de Curso	13
10.2	Aprendizajes del equipo	13
10.3	Limitaciones identificadas	13
A	Tabla ampliada de correspondencia modelo–código	16
B	Fragmentos representativos del código generado	16
C	Evidencias complementarias	16
D	Instrucciones detalladas de reproducción	17
E	Evidencia de contribuciones en GitHub	17

Resumen

Este informe presenta la aplicación del Desarrollo Dirigido por Modelos (MDD) al Sistema de Gestión para un Gimnasio. Se seleccionó el subsistema de clientes, membresías y pagos por integrar el registro de clientes, la gestión de planes, la vigencia de membresías y el control de pagos. El modelo UML de origen se construirá en StarUML con clases, atributos, operaciones, relaciones y cardinalidades. Mediante la extensión oficial para C# se generará la estructura del código, que será verificada por compilación y por la ejecución de una unidad mínima. Finalmente, se efectuará una regeneración controlada para comprobar que un cambio del modelo se refleje en el código. El repositorio público conservará el archivo nativo .mdj, la configuración, el código, las evidencias y las contribuciones individuales del equipo.

Palabras clave: MDD, StarUML, UML, generación de código, gimnasio.

1. Marco teórico

1.1. Model-Driven Engineering (MDE)

MDE considera los modelos como artefactos principales del desarrollo y los utiliza para producir código, configuraciones o documentación mediante transformaciones automatizadas. Su propósito es representar el dominio con mayor nivel de abstracción y reducir la dependencia de detalles tecnológicos [1, 2].

1.2. Model-Driven Architecture (MDA)

MDA es la propuesta del Object Management Group para separar la descripción funcional de un sistema de su plataforma de implementación. Esta separación se organiza mediante los niveles CIM, PIM y PSM y facilita la evolución hacia diferentes tecnologías [3].

1.3. Model-Driven Development (MDD)

MDD aplica los principios de MDE al ciclo de desarrollo: el modelo orienta la construcción y permite generar total o parcialmente el código. La calidad del resultado depende de la precisión del modelo, de las reglas de transformación y de la capacidad del generador [2].

1.4. Niveles de abstracción

1.4.1. Computation Independent Model (CIM)

El CIM describe objetivos, actores y procesos del negocio sin representar la estructura interna del software. En el PFC comprende el registro de clientes, la contratación de membresías y el control de pagos.

1.4.2. Platform Independent Model (PIM)

El PIM representa la estructura y el comportamiento sin asociarlos con un lenguaje o plataforma. Las clases `Cliente`, `Membresia` y `Pago` constituyen ejemplos de este nivel [4].

1.4.3. Platform Specific Model (PSM)

El PSM incorpora decisiones de implementación, como tipos de `C#`, espacios de nombres y convenciones de `.NET`. Es el modelo más próximo al código generado.

1.5. Transformaciones de modelos

1.5.1. Transformaciones modelo a modelo (M2M)

Las transformaciones M2M producen un modelo destino desde otro modelo, por ejemplo, de PIM a PSM. QVT define mecanismos estándar para relacionar y transformar modelos conformes con MOF [5].

1.5.2. Transformaciones modelo a texto (M2T)

Las transformaciones M2T generan artefactos textuales a partir de un modelo. La producción de archivos `.cs` desde las clases UML es una aplicación de este tipo de transformación, estandarizada por MOFM2T [6].

1.6. Metamodelos

Un metamodelo define los elementos y reglas de un lenguaje de modelado. UML se estructura sobre MOF; por ello, el generador puede interpretar clases, atributos, operaciones, asociaciones y enumeraciones de manera consistente [4].

1.7. Perfiles UML, estereotipos y valores etiquetados

Los perfiles amplían UML sin alterar su metamodelo. Los estereotipos expresan funciones específicas y los valores etiquetados almacenan propiedades adicionales que un generador puede utilizar. En este trabajo se priorizarán los elementos UML estándar compatibles con la extensión seleccionada.

1.8. Plantillas de generación

Una plantilla establece cómo se transforma cada elemento del modelo en texto. Puede definir, por ejemplo, la declaración de una clase, el tipo de un atributo o la representación de una relación múltiple mediante una colección.

1.9. Forward, reverse y round-trip engineering

La ingeniería directa genera código desde el modelo; la inversa reconstruye un modelo desde código existente. El roundtrip busca mantener la correspondencia entre ambos. La demostración aplicará ingeniería directa y una regeneración controlada para evidenciar el cambio modelo-código.

1.10. Diagramas UML como origen de generación

El diagrama de clases es el origen obligatorio porque contiene los elementos estructurales transformables. Otros diagramas solo se incorporan cuando aportan contexto o comportamiento relevante.

1.11. Estado del arte de las herramientas de transformación

La literatura registra numerosas herramientas MDD y retos relacionados con interoperabilidad, escalabilidad y adopción industrial [7, 8]. Por ello, la herramienta debe evaluarse por la trazabilidad y reproducibilidad que ofrece en el contexto concreto del PFC.

2. Justificación de la selección de StarUML

2.1. Requisitos y restricciones tecnológicas del PFC

El PFC corresponde a una aplicación de escritorio para gestionar un gimnasio, con C#, .NET, Windows Forms, SQL Server Express y Dapper. La herramienta CASE debe modelar UML, conservar el archivo nativo, generar código C# compilable y permitir repetir la generación en Windows [9].

2.2. Criterios de selección de la herramienta

La selección consideró compatibilidad con UML y C#, disponibilidad del generador, trazabilidad modelo-código, facilidad de instalación, licencia, curva de aprendizaje y posibilidad de almacenar todos los artefactos en GitHub.

2.3. Compatibilidad de StarUML con el lenguaje y la plataforma objetivo

StarUML admite diagramas de clases y amplía sus funciones mediante extensiones [10]. La extensión de C# transforma paquetes, clases, atributos, operaciones, enumeraciones y relaciones en archivos .cs [11]. El resultado será la capa estructural del dominio; Windows Forms, Dapper y SQL Server se completarán manualmente.

2.4. Licencia y disponibilidad

StarUML se distribuye para Windows y dispone de descarga y licencia comercial documentadas en su sitio oficial [12, 13]. La extensión de C# usa licencia MIT. El equipo registrará la versión realmente instalada y las condiciones empleadas durante la práctica.

2.5. Curva de aprendizaje

La interfaz gráfica, el modelado mediante elementos UML y el menú de extensión permiten aprender el flujo básico en poco tiempo. La mayor atención se concentrará en definir correctamente tipos, visibilidad, firmas, cardinalidades y directorio de salida.

2.6. Comparación con una herramienta alternativa

Cuadro 1: Comparación resumida de herramientas

Criterio	StarUML	Visual Paradigm
Generación C#	Mediante extensión oficial.	Mediante funciones de ingeniería de código.
Archivo nativo	Proyecto .mdj.	Proyecto .vpp.
Uso en la práctica	Flujo directo y suficiente para el alcance.	Mayor amplitud funcional y configuración.
Evidencia	Modelo, extensión y salida en GitHub.	Modelo y salida generada.

Visual Paradigm también ofrece generación y recuperación de código [14]; sin embargo, StarUML cubre el alcance estructural del subsistema con un flujo más breve para la demostración.

2.7. Decisión de selección

Se seleccionó StarUML porque satisface el modelado UML, la generación C# y la trazabilidad requerida, y permite entregar el archivo .mdj, la configuración y el código. La decisión se limita al objetivo académico de demostrar el ciclo modelo-código y no implica que genere la aplicación completa del gimnasio.

3. Descripción del subsistema del PFC

3.1. Contexto del Sistema de Gestión para un Gimnasio

El PFC busca centralizar la información operativa del gimnasio y sustituir registros dispersos de clientes, planes, membresías y pagos. La solución completa contempla interfaz de escritorio, reglas de negocio y persistencia; la demostración MDD se concentra en su estructura de dominio.

3.2. Descripción del subsistema de gestión de clientes, membresías y pagos

El subsistema administra los datos del cliente, los planes disponibles, la contratación o renovación de membresías y los pagos asociados. Fue elegido porque conecta varias entidades y reglas esenciales del PFC, por lo que resulta representativo para comprobar la generación de código.

3.3. Actores relacionados

Intervienen tres actores: el **administrador**, que configura planes y supervisa operaciones; el **empleado**, que registra clientes, membresías y pagos; y el **cliente**, cuyos datos y transacciones son gestionados por el sistema.

3.4. Requisitos funcionales relevantes

Cuadro 2: Requisitos funcionales seleccionados

Código	Descripción resumida
RF-SUB-01	Registrar y consultar la información de los clientes.
RF-SUB-03	Administrar planes con nombre, duración, precio y estado.
RF-SUB-04	Asignar un plan a un cliente mediante una membresía.
RF-SUB-06	Renovar la membresía y actualizar su vigencia.
RF-SUB-07	Registrar pagos vinculados con cliente, membresía y usuario.
RF-SUB-08	Emitir el comprobante de un pago confirmado.

3.5. Procesos principales

El flujo inicia con el registro del cliente, continúa con la selección del plan y la creación o renovación de la membresía, y finaliza con el registro del pago y la emisión del comprobante. Estos procesos justifican las relaciones entre las clases del modelo.

3.6. Alcance del subsistema

El modelo abarcará clientes, planes, membresías, pagos, detalles, comprobantes, usuarios y sus enumeraciones. Se generarán las clases C# y se verificará su correspondencia con los elementos UML.

3.7. Exclusiones del subsistema

Quedan fuera de la generación la interfaz Windows Forms, autenticación, inventario, rutinas, reportes, caja, base de datos, consultas SQL y repositorios Dapper. Estas funciones pertenecen al PFC, pero no al alcance estructural de la demostración.

4. Modelo UML de origen

4.1. Descripción general del modelo

El modelo representa el subsistema de clientes, membresías y pagos y sirve como entrada para generar código C#. Incluye nombres, tipos, visibilidad, firmas, relaciones y cardinalidades de acuerdo con UML 2.5.1 [4]. Por incorporar tipos de C# y decisiones de .NET, se considera próximo a un PSM.

4.2. Diagrama de clases

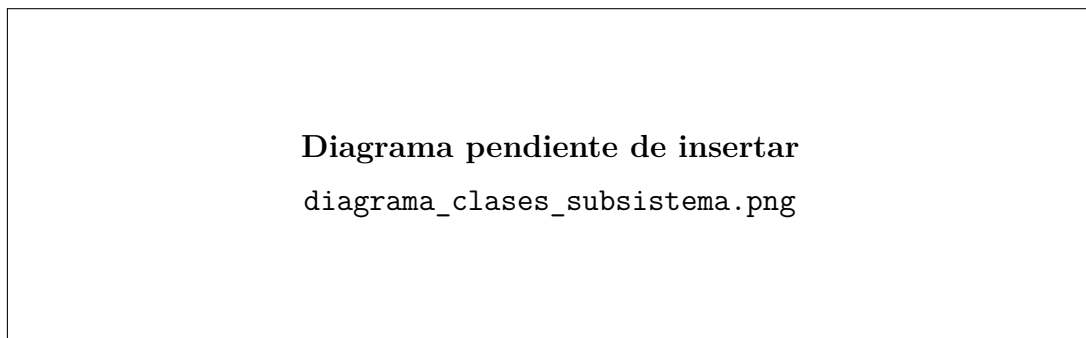


Figura 1: Diagrama de clases del subsistema modelado

4.3. Clases del dominio

El modelo contiene siete clases: `Cliente`, `PlanMembresia`, `Membresia`, `Pago`, `DetallePago`, `ComprobantePago` y `Usuario`. Además, emplea `EstadoMembresia`, `MetodoPago`, `EstadoPago` y `RolUsuario` para restringir estados, métodos y roles. Las responsabilidades detalladas se conservan en el Anexo A.

4.4. Atributos y operaciones

Las clases incluyen identificadores y datos propios del dominio, junto con operaciones de actualización, activación, renovación, cálculo, confirmación y anulación. Todos los atributos tendrán tipo y visibilidad, y las operaciones registrarán parámetros y retorno completos. La tabla ampliada se presenta en el Anexo A para evitar duplicar el contenido visible en el diagrama.

4.5. Relaciones y cardinalidades

Un cliente puede tener varias membresías y pagos; cada membresía corresponde a un plan; un usuario registra múltiples pagos; un pago se compone de uno o varios detalles y puede producir un comprobante. Se utilizarán asociaciones o composiciones solo cuando su significado sea coherente con el dominio, sin forzar herencias innecesarias.

4.6. Estereotipos y perfiles utilizados

El modelo utilizará inicialmente los elementos estándar de UML, compatibles con la extensión de C# [11]. Solo se documentará un estereotipo adicional si aparece realmente en el archivo `.mdj` y afecta la generación.

4.7. Validación sintáctica y semántica del modelo

La revisión comprobará nombres únicos, tipos, visibilidad, firmas completas, roles, cardinalidades y ausencia de elementos aislados. También verificará que las relaciones representen los requisitos del subsistema.

Resultado pendiente: modelo validado en StarUML [VERSIÓN] el [FECHA], con [CANTIDAD] errores y [CANTIDAD] observaciones.

4.8. Archivo nativo del modelo en el repositorio

El archivo se almacenará en `modelo/modelo_subsistema_clientes_membresias_pagos.mdj`. El acceso directo estará disponible en [el repositorio público del equipo](#).

5. Configuración del generador

5.1. Generador o extensión utilizada

Se utilizará *C# Extension for StarUML*, que interpreta el modelo `.mdj` y genera archivos `.cs`. La ejecución se realizará sobre el paquete base del subsistema y no sobre elementos aislados.

5.2. Origen y versión del generador

La extensión oficial corresponde al paquete `staruml.csharp`, versión 0.9.7, publicado por Dongjoon Lee con licencia MIT y compatibilidad declarada con StarUML 6 o superior [11]. Se registrará también la versión exacta de StarUML instalada.

5.3. Lenguaje y plataforma objetivo

El lenguaje objetivo es C# y la plataforma de comprobación será .NET sobre Windows. El código generado representará el dominio y posteriormente podrá integrarse con Windows Forms, SQL Server Express y Dapper.

5.4. Parámetros de generación

Cuadro 3: Configuración resumida del generador

Parámetro	Valor
Paquete base	GestionClientesMembresiasPagos
Espacio de nombres	Gimnasio.Dominio
Lenguaje	C#
Directorio de salida	generado/src/Gimnasio.Dominio/
Convención	PascalCase para tipos; camelCase para miembros privados.
Sobrescritura	Permitida solo en el directorio generado.

5.5. Directorio de salida y convenciones de nombres

Cada clase y enumeración se almacenará en un archivo del mismo nombre. Los archivos manuales de prueba se ubicarán en `generado/verificacion/`, fuera del directorio reemplazable.

5.6. Política ante la regeneración

Antes de regenerar se guardará el modelo y se realizará un commit. El directorio `generado/src/Gimnasio.Dominio/` podrá reemplazarse por completo; por tanto, no contendrá cambios manuales. Esta política facilita comparar el resultado anterior y posterior.

5.7. Decisiones de mapeo UML a código

Los paquetes se mapearán a espacios de nombres; clases y enumeraciones a tipos C#; atributos a campos; operaciones a métodos; generalizaciones a herencia; y multiplicidades múltiples a colecciones. La tabla ampliada de correspondencia se encuentra en el Anexo A.

5.8. Archivos de configuración en el repositorio

La carpeta `plantillas/` conservará `configuracion_generador.md`, `mapeo_uml_cs_harp.md` y los archivos oficiales de la extensión necesarios para identificar su origen y versión. Esto permitirá repetir el proceso sin depender únicamente de las capturas.

6. Proceso de generación

6.1. Preparación del entorno

La prueba se realizará en Windows con StarUML, la extensión de C# y el SDK de .NET instalados. Las versiones reales se registrarán antes de la demostración.

Cuadro 4: Entorno utilizado para la generación

Componente	Versión comprobada
Sistema operativo	[COMPLETAR]
StarUML	[COMPLETAR]
Extensión C#	0.9.7
SDK de .NET	[COMPLETAR]

6.2. Apertura y selección del modelo de origen

Se abrirá el archivo `modelo_subsistema_clientes_membresias_pagos.mdj`, se comprobará el diagrama y se seleccionará el paquete `GestionClientesMembresiasPagos`. El modelo se guardará antes de invocar el generador.

6.3. Ejecución del generador

El procedimiento será: **1)** abrir Tools; **2)** elegir C# > Generate Code...; **3)** seleccionar el paquete base; y **4)** indicar `generado/src/Gimnasio.Dominio/` como salida [11].

6.4. Registro o log de salida

El mensaje final, la fecha, la hora y cualquier advertencia se guardarán en `evidencias/logs/generacion_csharp.txt`. Si la extensión no produce un log exportable, se transcribirá fielmente la salida visible y se adjuntará la captura.

6.5. Árbol del proyecto generado

El árbol deberá mostrar los archivos de las siete clases y cuatro enumeraciones dentro del espacio de nombres del dominio. Los archivos `Program.cs` y `VerificacionGenerado.csproj` se ubicarán en la carpeta manual de verificación y no se contabilizarán como generados.

6.6. Artefactos producidos

Se registrará el número real de clases, enumeraciones y demás archivos producidos. StarUML no deberá acreditarse como origen de formularios, repositorios, base de datos o archivos creados manualmente.

6.7. Tiempo de generación

El tiempo se medirá desde la confirmación del directorio de salida hasta el mensaje final: [FECHA], de [HORA INICIAL] a [HORA FINAL], con una duración de [SEGUNDOS] segundos y [CANTIDAD] archivos generados.

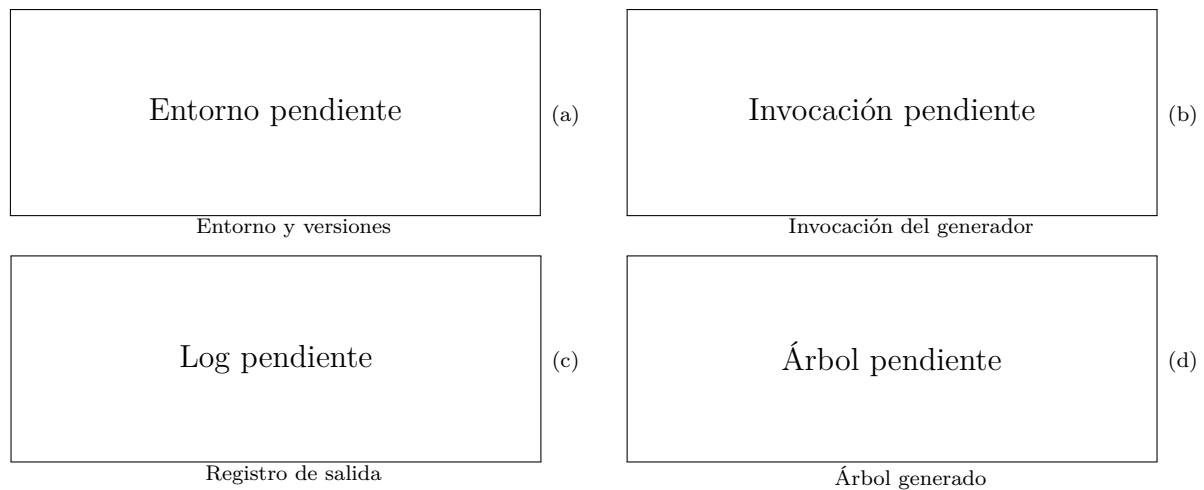


Figura 2: Evidencias principales del proceso de generación

7. Verificación del código generado

La verificación comprobará la estructura, sintaxis, compilación, ejecución y fidelidad del código. Los resultados definitivos se completarán únicamente después de ejecutar la práctica.

7.1. Estructura del proyecto generado

Se verificará la presencia de [CANTIDAD] archivos `.cs`: [CANTIDAD] clases y [CANTIDAD] enumeraciones, almacenados en `generado/src/Gimnasio.Dominio/`. Los archivos manuales permanecerán en `generado/verificacion/`.

7.2. Verificación sintáctica

La revisión comprobará espacios de nombres, tipos, firmas, llaves, referencias y colecciones. El resultado final registrará [CANTIDAD] errores y [CANTIDAD] advertencias, junto con las correcciones realizadas.

7.3. Compilación del código

Como StarUML genera archivos fuente y no una solución ejecutable completa, se usará un proyecto de consola manual que incluya las clases sin modificarlas.

Listing 1: Comandos de verificación

```
cd generado/verificacion
dotnet restore
dotnet build
dotnet run
```

Resultado: SDK [VERSIÓN], [ERRORES] errores, [ADVERTENCIAS] advertencias y estado [EXITOSO O FALLIDO].

7.4. Ejecución de una unidad mínima demostrable

El archivo manual `Program.cs` instanciará tipos generados e invocará una operación pública sin alterar el directorio de salida. La prueba demostrará que el código puede integrarse y ejecutarse en .NET, pero no se presentará como la implementación completa de las reglas del gimnasio.

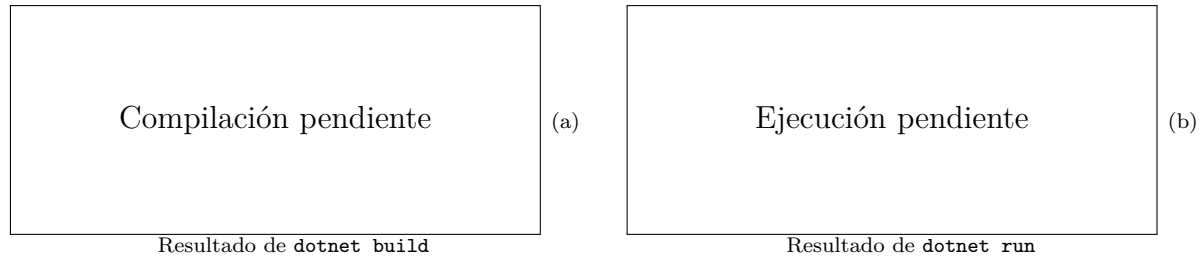


Figura 3: Compilación y ejecución del código generado

7.5. Correspondencia entre modelo y código

Cuadro 5: Correspondencia representativa modelo-código

Elemento UML	Código esperado	Resultado real
Clase <code>Cliente</code>	<code>Cliente.cs</code> y declaración de clase.	[COMPLETAR]
<code>idCliente:int</code>	Campo de tipo <code>int</code> .	[COPIAR DECLARACIÓN]
<code>cambiarEstado</code>	Método público con retorno <code>void</code> .	[COPIAR FIRMA]
<code>EstadoMembresia</code>	Archivo y declaración <code>enum</code> .	[COMPLETAR]
<code>Pago-DetallePago</code>	Referencia o colección de detalles.	[COPIAR TIPO]

La correspondencia completa de clases, atributos, operaciones y relaciones se traslada al Anexo A.

7.6. Diferenciación entre código generado y código manual

Los archivos de `generado/src/` serán producidos por StarUML y no se editarán. `Program.cs`, el proyecto `.csproj` y los logs serán manuales y estarán separados. Esta organización impide presentar como generado un artefacto creado por el equipo.

7.7. Limitaciones del generador

La extensión genera principalmente estructura y presenta estas limitaciones:

- no implementa reglas de negocio ni cuerpos funcionales completos;
- no produce Windows Forms, Dapper, SQL Server ni la solución final;
- las asociaciones y colecciones deben revisarse después de generar;
- los cambios manuales dentro de la salida pueden perderse al regenerar.

Estas limitaciones no invalidan la prueba: delimitan qué automatiza StarUML y qué deberá completarse manualmente en el PFC.

8. Cierre del ciclo y roundtrip controlado

8.1. Estado inicial del modelo y del código

Se registrará el commit inicial, el elemento UML seleccionado y su declaración en el código antes del cambio: **[COMPLETAR CON COMMIT Y ELEMENTO]**.

8.2. Cambio realizado en el modelo UML

Se modificará un elemento pequeño y verificable, por ejemplo, la incorporación del atributo `fechaRegistro: DateTime` en `Cliente`. El cambio exacto se documentará sin editar previamente el código generado.

8.3. Regeneración del código

Después de guardar el `.mdj`, se ejecutará nuevamente el mismo generador, paquete base, directorio y política de sobrescritura descritos en la Sección 6.

8.4. Evidencia antes y después

La evidencia combinará una captura del modelo antes y después y la diferencia del archivo `C#` correspondiente. Los archivos se guardarán en `evidencias/roundtrip/`.

8.5. Trazabilidad modelo a código

La prueba será satisfactoria si el cambio aparece en el archivo esperado con nombre, tipo y visibilidad equivalentes: **[REGISTRAR RESULTADO REAL]**.

8.6. Manejo del código manual ante la regeneración

El proyecto manual de verificación permanecerá fuera del directorio sobrescrito. Después de regenerar se repetirá `dotnet build` para confirmar que el cambio no afectó su compilación.

9. Reparto del trabajo

9.1. Roles y responsabilidades

El equipo distribuirá cuatro responsabilidades diferenciadas: modelado UML, configuración y generación, verificación técnica, y documentación y presentación. Los nombres y roles definitivos deberán coincidir con la portada.

9.2. Tareas realizadas por integrante

Cada integrante registrará tareas concretas y verificables, evitando descripciones generales como “ayudó en todo”. La tabla definitiva se completará con el modelo, código, informe y evidencias realmente aportados.

9.3. Commits atribuibles a cada integrante

Se incluirá al menos un commit sustantivo de cada integrante con su mensaje, fecha y enlace directo: **[COMPLETAR CON CUATRO O MÁS COMMITS REALES]**.

9.4. Log de aportaciones de GitHub

El historial completo se conservará en el Anexo E. Los commits deberán usar la cuenta personal de cada integrante para que la autoría sea identificable.

9.5. Participación en la exposición y demostración

Los cuatro integrantes explicarán una parte diferenciada y participarán en la demostración: teoría y justificación; modelo; generación y roundtrip; y verificación, resultados y conclusiones.

10. Conclusiones

10.1. Valor de MDD para el Proyecto Fin de Curso

MDD aporta una relación verificable entre el modelo del subsistema y la estructura inicial del código. En el PFC puede reducir tareas repetitivas y mantener nombres y tipos coherentes, siempre que el modelo se construya con precisión y se controle la regeneración.

10.2. Aprendizajes del equipo

La actividad permite diferenciar un diagrama usado solo como documentación de un modelo que actúa como entrada de un generador. También evidencia la importancia de registrar versiones, parámetros, resultados y commits para que otra persona pueda repetir el proceso.

10.3. Limitaciones identificadas

StarUML genera principalmente la estructura del dominio y no reemplaza el desarrollo de interfaces, persistencia ni reglas de negocio. Las conclusiones definitivas deberán actualizarse después de la práctica con los resultados reales de compilación, ejecución y roundtrip.

Referencias bibliográficas

- [1] D. C. Schmidt, “Guest Editor’s Introduction: Model-Driven Engineering,” *Computer*, vol. 39, no. 2, pp. 25–31, Feb. 2006, doi: 10.1109/MC.2006.58.
- [2] B. Selic, “The Pragmatics of Model-Driven Development,” *IEEE Software*, vol. 20, no. 5, pp. 19–25, Sep./Oct. 2003, doi: 10.1109/MS.2003.1231146.
- [3] Object Management Group, *Model Driven Architecture Guide, Revision 2.0*. Needham, MA, USA: OMG, Jun. 2014. [En línea]. Disponible en: <https://www.omg.org/cgi-bin/doc?ormsc/14-06-01>
- [4] Object Management Group, *OMG Unified Modeling Language, Version 2.5.1*. Needham, MA, USA: OMG, Dec. 2017. [En línea]. Disponible en: <https://www.omg.org/spec/UML/2.5.1>
- [5] Object Management Group, *Meta Object Facility Query/View/Transformation, Version 1.3*. Needham, MA, USA: OMG, Jun. 2016. [En línea]. Disponible en: <https://www.omg.org/spec/QVT/1.3>
- [6] Object Management Group, *MOF Model to Text Transformation Language, Version 1.0*. Needham, MA, USA: OMG, Jan. 2008. [En línea]. Disponible en: <https://www.omg.org/spec/MOFM2T/1.0>
- [7] N. Kahani, M. Bagherzadeh, J. R. Cordy, J. Dingel and D. Varró, “Survey and classification of model transformation tools,” *Software and Systems Modeling*, vol. 18, no. 4, pp. 2361–2397, 2019, doi: 10.1007/s10270-018-0665-6.
- [8] A. Bucchiarone, J. Cabot, R. F. Paige and A. Pierantonio, “Grand challenges in model-driven engineering: An analysis of the state of the research,” *Software and Systems Modeling*, vol. 19, no. 1, pp. 5–13, 2020, doi: 10.1007/s10270-019-00773-6.
- [9] Microsoft, “C# documentation,” *Microsoft Learn*, 2026. [En línea]. Disponible en: <https://learn.microsoft.com/en-us/dotnet/csharp/> [Consultado: 16-jul-2026].
- [10] MKLabs Co., Ltd., “StarUML: A sophisticated software modeler for agile and concise modeling,” *StarUML*, 2026. [En línea]. Disponible en: <https://staruml.io/> [Consultado: 16-jul-2026].
- [11] StarUML, “C# Extension for StarUML,” versión 0.9.7, GitHub, 2025. [En línea]. Disponible en: <https://github.com/staruml/staruml-csharp> [Consultado: 16-jul-2026].
- [12] MKLabs Co., Ltd., “Download and start now,” *StarUML*, 2026. [En línea]. Disponible en: <https://staruml.io/download> [Consultado: 16-jul-2026].

- [13] MKLabs Co., Ltd., “Pricing,” *StarUML*, 2026. [En línea]. Disponible en: <https://staruml.io/pricing/> [Consultado: 16-jul-2026].
- [14] Visual Paradigm International, “UML/Code Generation Tool,” *Visual Paradigm*, 2026. [En línea]. Disponible en: <https://www.visual-paradigm.com/features/code-engineering-tools/> [Consultado: 16-jul-2026].

A. Tabla ampliada de correspondencia modelo–código

Este anexo conserva el detalle retirado del cuerpo para cumplir el límite de páginas sin perder la definición técnica del modelo.

Cuadro 6: Atributos y operaciones principales del dominio

Clase	Atributos	Operaciones
Cliente	idCliente, cedula, nombres, apellidos, telefono, correo, activo	actualizarDatos(), cambiarEstado()
PlanMembresia	idPlan, nombre, duracionDias, precio, activo	calcularVencimiento(), cambiarEstado()
Membresia	idMembresia, fechaInicio, fechaFin, estado, precioAcordado	activar(), renovar(), estaVigente()
Pago	idPago, fechaPago, total, metodo, estado	agregarDetalle(), calcularTotal(), confirmar(), anular()
DetallePago	idDetalle, concepto, cantidad, valorUnitario, subtotal	calcularSubtotal()
ComprobantePago	idComprobante, numero, fechaEmision, total	emitir(), obtenerResumen()
Usuario	idUsuario, nombreUsuario, rol, activo	puedeRegistrarPagos(), cambiarEstado()

Cuadro 7: Reglas de mapeo UML a C#

Elemento UML	Resultado C#	Criterio de comprobación
Paquete	Espacio de nombres y carpeta.	Coincidencia del nombre base.
Clase	Clase y archivo .cs.	Una declaración por clase.
Atributo	Campo con tipo y visibilidad.	Equivalencia de nombre y tipo.
Operación	Método con parámetros y retorno.	Firma completa.
Enumeración	Tipo enum.	Valores del modelo conservados.
Multiplicidad múltiple	Colección tipada.	Tipo y cardinalidad revisados.
Generalización	Herencia mediante dos puntos.	Clase base correcta.

B. Fragmentos representativos del código generado

Se incorporarán únicamente fragmentos obtenidos durante la práctica: una clase, una enumeración, una operación y una relación. Cada listado indicará el archivo de origen y no incluirá código manual presentado como generado.

C. Evidencias complementarias

Este anexo reunirá las capturas individuales del entorno, invocación, log, árbol, compilación, ejecución y roundtrip, además de los archivos de registro. En el cuerpo se mantendrán las composiciones necesarias para la evaluación.

D. Instrucciones detalladas de reproducción

1. clonar el repositorio y abrir el modelo `.mdj`;
2. comprobar las versiones de StarUML y de la extensión C#;
3. ejecutar el generador con los parámetros documentados;
4. abrir `generado/verificacion/`;
5. ejecutar `dotnet restore`, `dotnet build` y `dotnet run`;
6. compilar este informe con PdfLaTeX dos veces.

E. Evidencia de contribuciones en GitHub

Cuadro 8: Registro ampliado de contribuciones

Integrante	Aporte verificable	Fecha	Enlace al commit
Díaz Pontón Steven Santiago	[COMPLETAR]	[FECHA]	[URL]
Escudero Plaza María del Rosario	[COMPLETAR]	[FECHA]	[URL]
Mera Arias Erick Jhair	[COMPLETAR]	[FECHA]	[URL]
Mesías Quijije Jhon Alexander	[COMPLETAR]	[FECHA]	[URL]